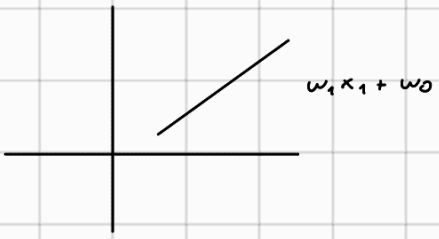
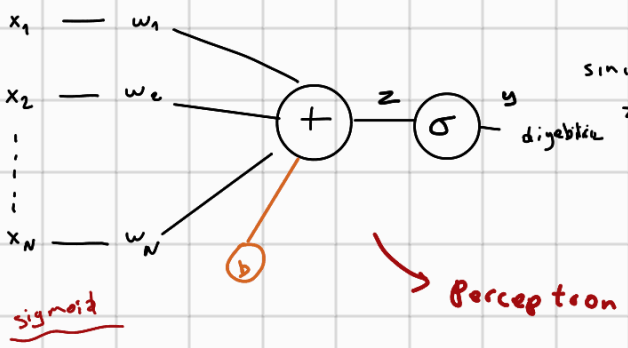
ROC curve *



$\vec{w} \cdot \vec{x} + b = \hat{r}^t$

$\mathcal{L}(\theta | X) = \sum_{t=1}^{N} (\hat{r}^t - r^t)^2$ türevini alıp 0'a eşitlemiştik

başka yönden



sınıflandırma için

$z < 0 \Rightarrow C_0$

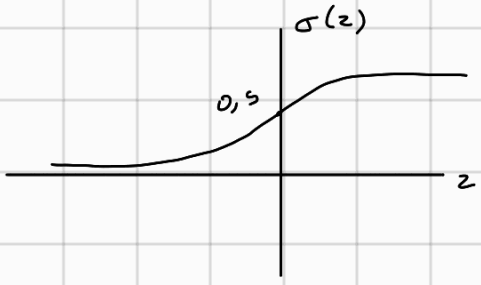
$z \geq 0 \Rightarrow C_1$

$P(C_0)$

Perceptron

sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



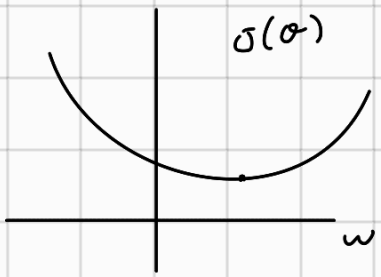
$z < 0 \Rightarrow C_0$

$z \geq 0 \Rightarrow C_1$

Bu şekilde iteratif olarak ilerliyoruz

gradient desent

$$w_i \leftarrow w_i - \eta \frac{\partial J}{\partial w_i}$$ → maliyet fonksiyonu



Back Propagation

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}, \quad z_i = w^T \cdot x_i + b$$

entropy $\sum_{i=1}^{k} p_i \log p_i$

$$J(w) = -\frac{1}{N} \sum_{i=0}^{N} \underbrace{y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)}_{\text{cross entropy}}$$

$$\frac{\partial J(w)}{\partial w} = \frac{\partial J(w)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w}$$

önceden (sonra bunu) (sonra bunu)

$$\frac{\partial J(w)}{\partial \hat{y}_i} = \frac{-y_i}{\hat{y}_i} + \frac{1-y_i}{1-\hat{y}_i} \qquad \frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1-\hat{y}_i)$$

$$\frac{\partial z_i}{\partial w} = x_i$$

$$\frac{\partial J(w)}{\partial w} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i) \, x$$

→ sonra bunu ekleyeceğiz

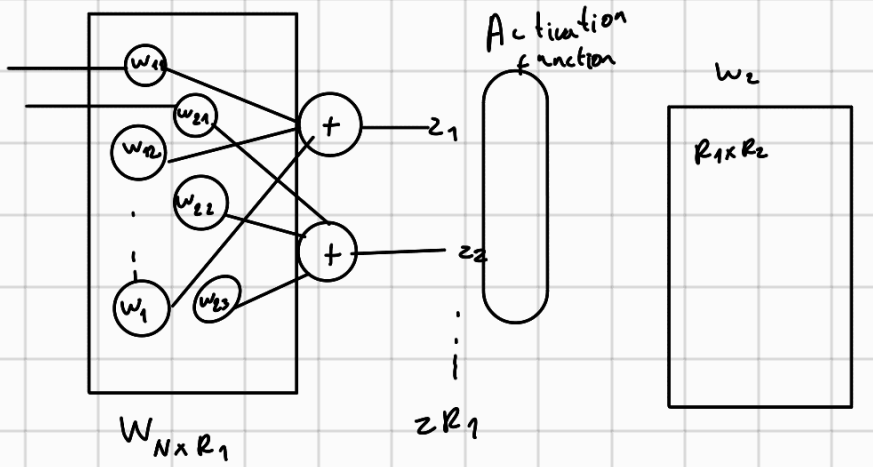
geri sarrakyapıyoruz

$$\frac{\partial J(w)}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$$

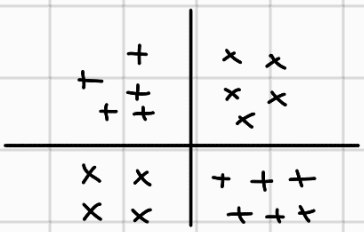
$$x = \{x^t, o^t\}_{t=1}^{M}$$

$$P(c_0) = \hat{y}_i$$

$$P(c_1) = (1-\hat{y}_i)$$



multilayer perceptron



activation function

$\Phi(wx+b)$

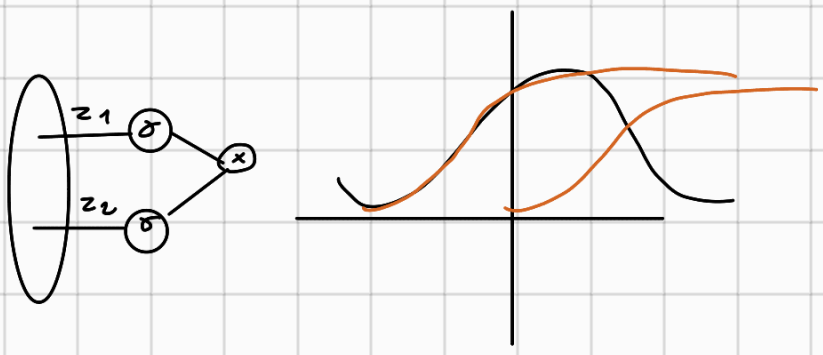
$$\phi_2\left(W \Sigma \phi_1(wx+b)\right)$$

Fourierde ← Basis function

ortogonal ↙

$$\langle \phi_1(x), \phi_2(x) \rangle = \int_{-\infty}^{\infty} \phi_1(x)\,\phi_2(x)\,dx = 0$$

ama burda



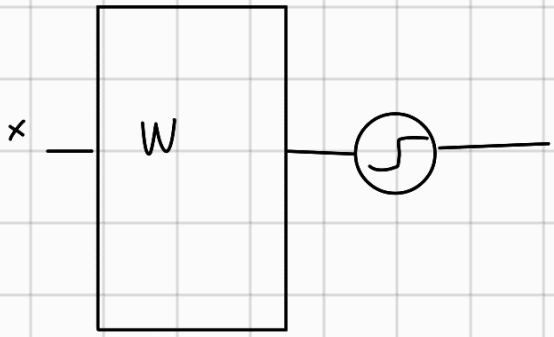
Biz ortogonal olmalarını beklemiyoruz

Böyle bir teorem var $\sum_{i=1}^{\infty} w_i \phi(x - x_i) \approx g(x)$

yanaşak baz fonksiyonlarıyla mükemmel olarak tahmin edebiliriz
teorik olarak

universal approximator

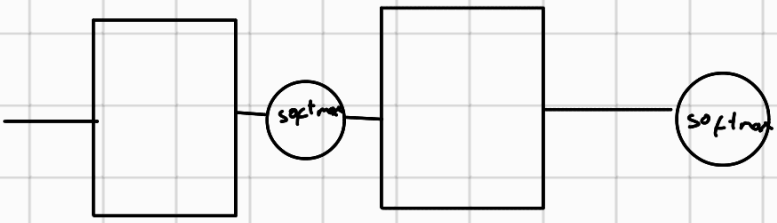
$x \rightarrow f(x) \rightarrow y$



① $$\text{softmax} = \frac{e^{z_i}}{\sum_{j=1}^{c} e^{z_j}}$$

↙ olasılık dağılımına ihtiyacımız olduğunda

soru



böyle bir sıkıntı var sonsuza giderken türev 0 o yüzden back propogation iyi çalışmaz

bu yüzden genelde softmax en son katmanda kullanılır

(2) ReLu

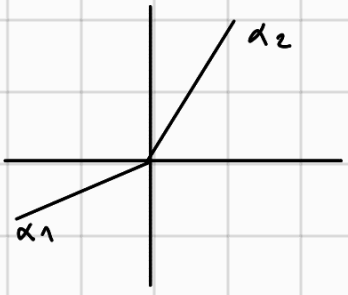


ax+b şeklinde gösteriyoruz

non linear

negatif tarafta yine türevi 0, dead neuron sorununa yol açıyor

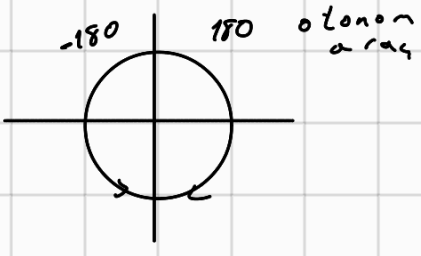
(3) Leeky ReLu



nonlinearity daha az olduğu için daha fazla katman gerekiyor

$180 \times [-1,1]$

(4) $\tanh = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$



olanan aralık

bunda da türevler $+\infty, -\infty$ da 0'a yaklaşıyor

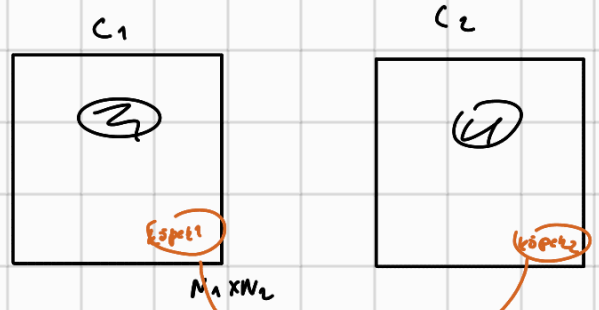
Derin öğrenmede katmanlar artar

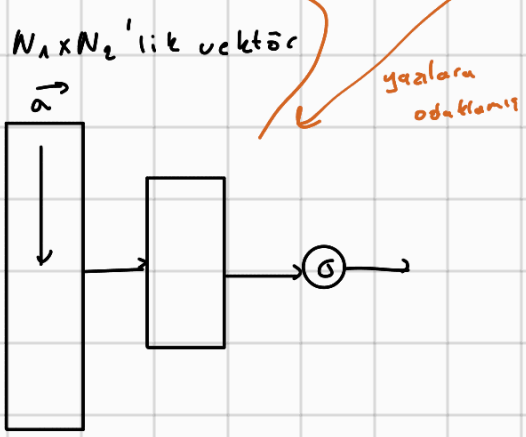
① Tenserflow

② Pytorch

Tensor Dataset

Data augmentation





Pseudo random

x kitleann

Data Scaler( ) → Standart Scaler( )

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad -1 < x_i < 1$$

$10^6 _ 10^{-4}$

$2 _ 10^{-4}$

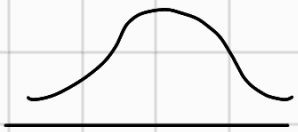
Scaler

① min - max

$\frac{x - x_{min}}{x_{max} - x_{min}}$

② Gauss $M(0,1)$

$\frac{x - \mu}{\sigma}$



sonra TensorDataset

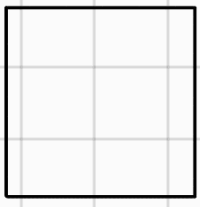
random split

DataLoader

class ANNmodel(nn.Model) ← bunları inherit ediyoruz

init : super().__init__()

pytorch linear torch.nn.Linear(in_features, out_features, bias=true, device=None, dtype=None)



parametre sayısı = N x M + N

↙ in out

net = nn.Sequential

BatchNorm1d

Dropout

optim.Adam

nn.CrossEntropyLoss

```
def forward(self,x)
    x = x.to(self.device)
    x = x.to(T.float32)
```

```
out_ = F.one_hot(T.argmax(out,dim=1), num_classes=64)
```

```
def learn:
```

```
self.optimizer.zero_grad
y_pred = self.forward(x)
loss = self.loss(y_pred, y)
loss.backward()
```

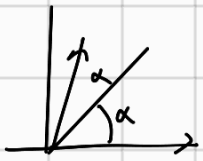
Normalization layer

BatchNorm1d

Dropout layers

overfit etmemek için bazı verileri düşürüyoruz

Cosined similiraty



$$\frac{x \cdot y}{||x||||y||} = \cos(\theta)$$

Loss Functions

- L1 loss
- MSE loss
- KLDivLoss